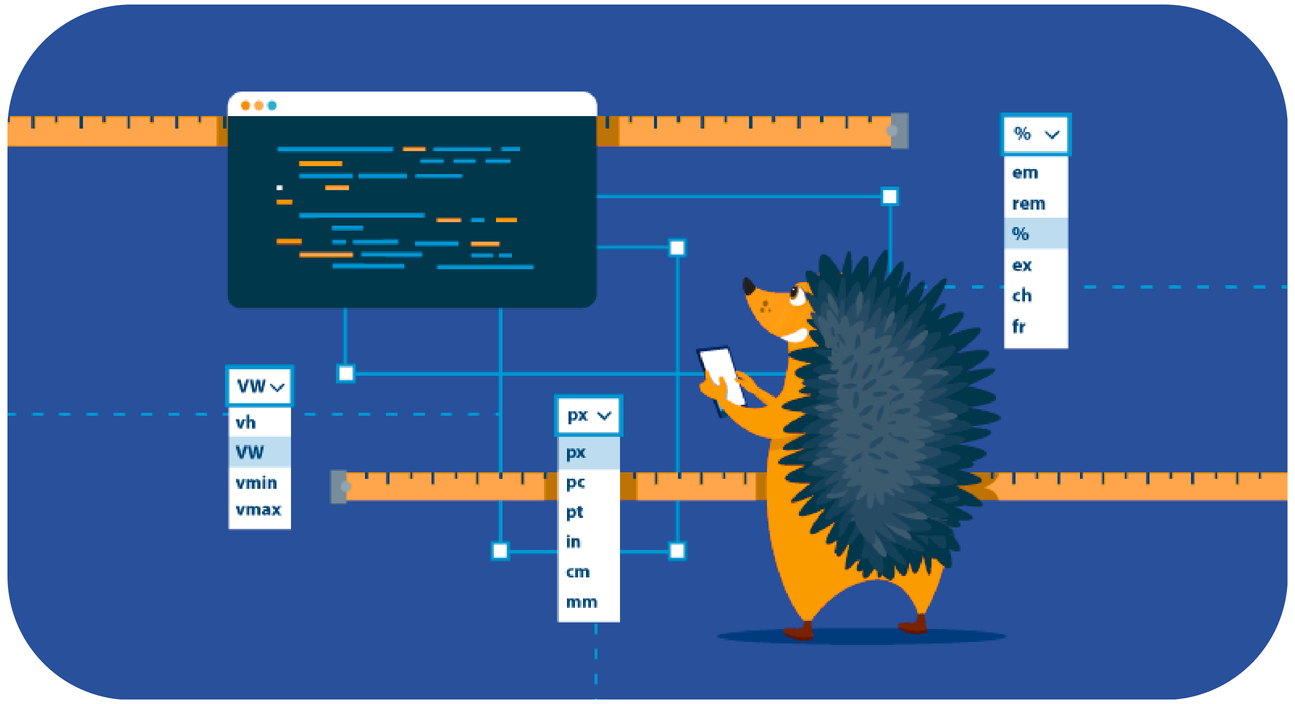
indiXpert

CSS Sizes, Units & Styling Images



CSS Sizes

CSS Height & Width

CSS provides the height and width properties to control the size of elements, affecting layout, responsiveness, and design consistency.

📏 Common Units

- px – fixed pixels
- % – relative to parent
- em / rem – relative to font size
- vh / vw – relative to viewport height/width

🧩 Special Values

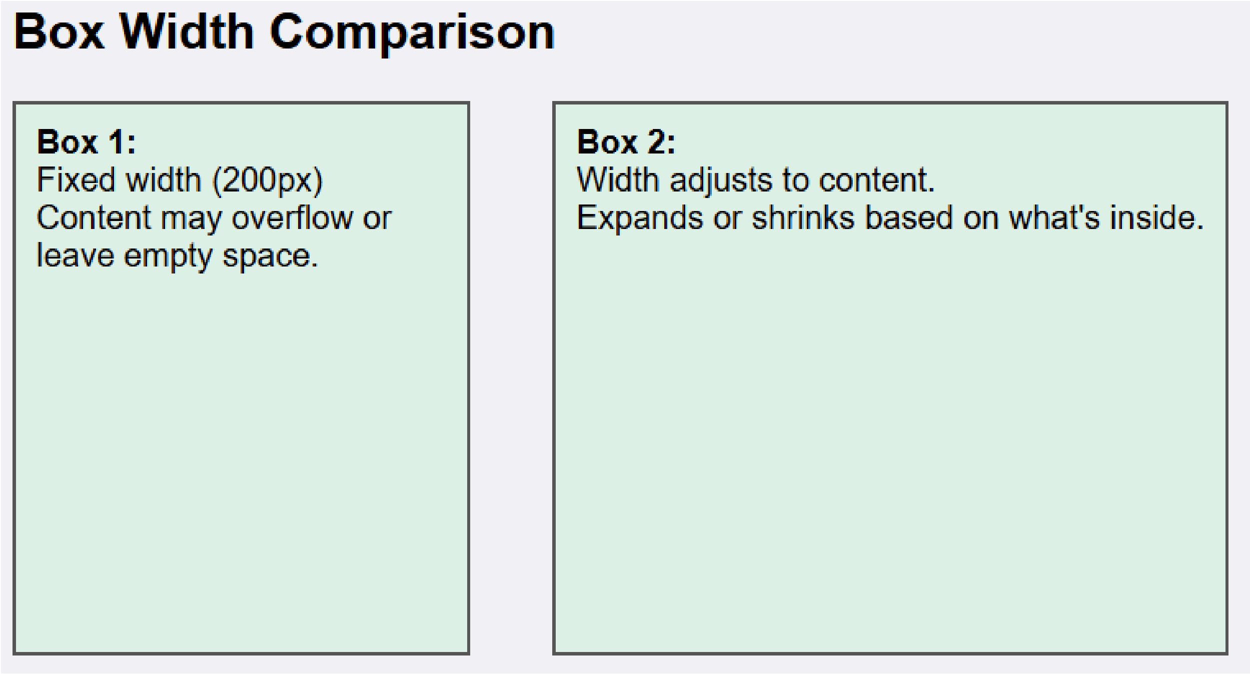
- auto – size is based on content or parent
- min-content – smallest content size without overflow
- max-content – expands to fit the longest unbroken content
- fit-content – fits content up to a max limit

Example:

```
<style>
    .box {
      width: 200px;
      height: 30vh;
  }

    .box {
      width: fit-content;
      height: 30vh;
  }
</style>
```

Output:



CSS UNITS

CSS provides various units to define the size of elements, spacing, and fonts. These units are categorized into absolute and relative types, and understanding them is key to building responsive and scalable layouts.

Absolute Units

- CSS units define element sizes, spacing, and fonts.
- Absolute units have a fixed size that doesn't change with screen size.
- Best used when exact sizing is needed (e.g., for print layouts).
- Not ideal for responsive web design.

Common Absolute Units

- **px (pixels):** Fixed screen dots. Most common for web.
- **cm / mm / in:** Centimeters, millimeters, inches – used in print.
- **pt (points):** 1pt = 1/72 inch.
- **pc (picas):** 1pc = 12pt. Rare in web use.

Example:

```
<style>
  .box {
    width: 10cm;
    height: 100px;
    background-color: #add8e6;
    padding: 12pt;
    font-size: 16px;
    border: 1px solid #333;
  }
</style>
```

CSS UNITS

indiXpert

Relative Units

Relative units adjust based on their context, like the parent element or viewport, making them ideal for building flexible and responsive layouts.

% (Percentage):

% sets size relative to the parent element.

```
<div style="width: 50%;">
  50% width of parent
</div>
```

vw / vh (Viewport Width / Height):

vw and vh set size relative to the viewport width and height.

```
<div style="width: 50vw;">
50vw width (relative to viewport)
</div>
```

em:

em is based on the font size of the parent element.

```
<div style="font-size: 20px;">
  Parent text
  <span style="font-size: 1.5em;">
    → 1.5em text (30px)
  </span>
</div>
```

rem (Root em):

rem is based on the font size of the root (html) element.

```
<p style="font-size: 2rem; color: #333;">
  2rem text = 32px
</p>
```

CSS UNITS

Time units are used to control durations in animations and transitions.

- ms: milliseconds (1s = 1000ms)
- s: seconds

Example:

```
<style>
  a {
    transition: all 400ms ease-in-out;
    }
  div {
     animation-duration: 2s;
    }
</style>
```

📐Choosing the Right Unit

Pick the unit based on what you're styling:

- **px** → Precise sizing (e.g. borders, buttons)
- **em / rem** → Responsive typography
- **%** → Relative to container
- **vw / vh** → Full-screen elements
- **s / ms** → Animations & transitions

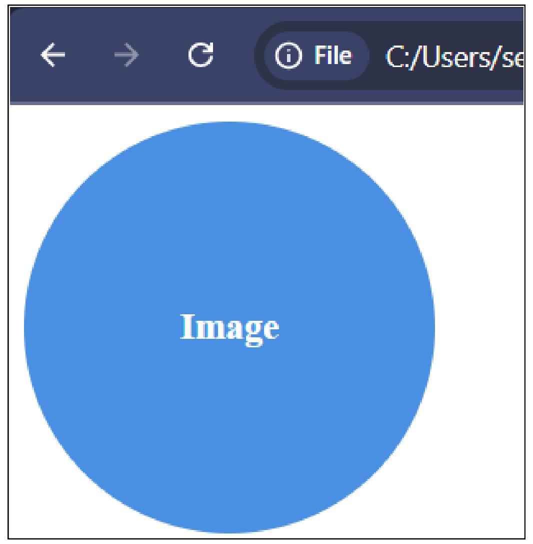
🔎 **For more advanced units like ch, ex, or fr, check W3C CSS Values and Units**

Styling Images

Rounded Images

```html
<body>
    <img src="..." class="rounded">
    <style>
      img.rounded { border-radius: 50%; }
    </style>
</body>
```

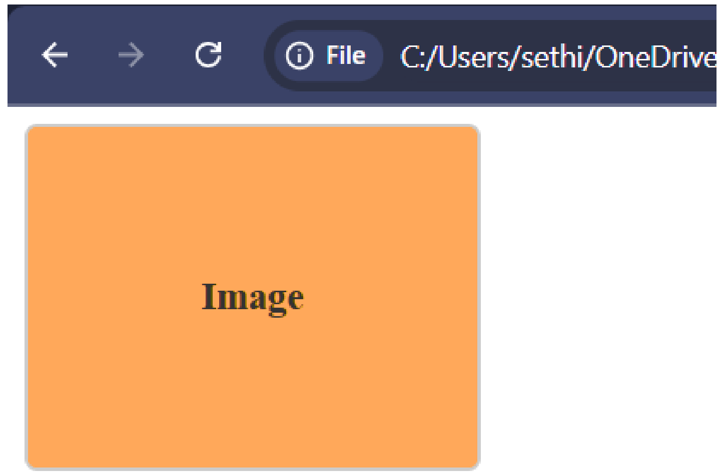
Syntax



Output

Thumbnail Images

```html
<body>
    <img src="..." class="thumb">
    <style>
      img.thumb {
        border: 1px solid #ccc;
        padding: 5px;
        border-radius: 5px;
      }
    </style>
</body>
```



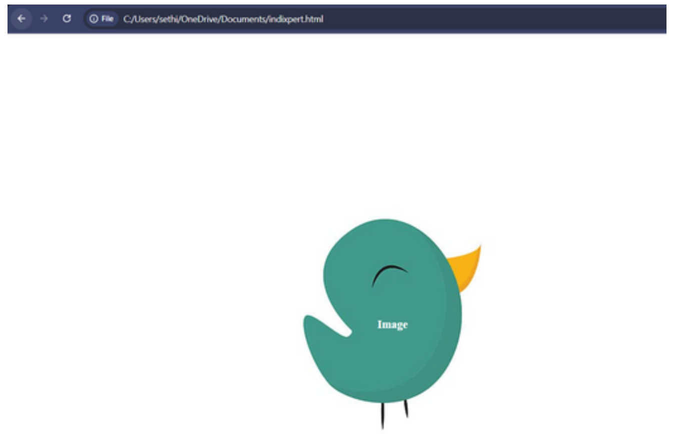
Output

Syntax

Responsive Images

```html
<body>
    <img src="..." class="responsive">
    <style>
      img.responsive { max-width: 100%; height: auto; }
    </style>
</body>
```

Syntax

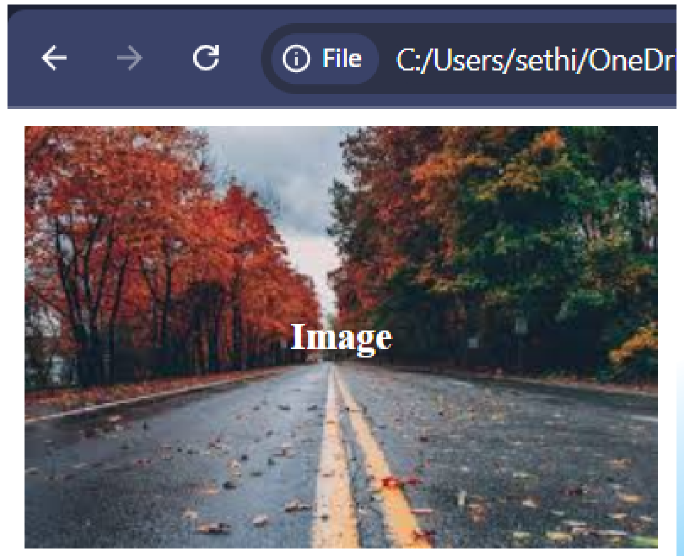


Output

Object Fit

```html
<body>
    <img src="..." class="cover">
    <img src="..." class="contain">
    <style>
      img.cover { object-fit: cover; }
      img.contain { object-fit: contain; }
    </style>
</body>
```

Syntax



Output

THANK YOU

PHONE NUMBER

(+91) 778 899 2897

WEBSITE

www.indixpert.com